

学海漫游 · 实验报告

《学海漫游：异世界的信科少女》 · 实验报告

北京大学信息科学技术学院 · 程序设计课程大作业

开发团队：代易瓚（剧情核心）· 闫瑾（界面核心）· 沈昱萱（玩法核心）

技术栈：C++17 · Qt 6.11 · CMake 3.16 · Git/GitHub · JSON 数据驱动

GitHub：look-dyz/StudyAdventure

摘要

本项目是一款以北京大学信息科学技术学院大一课程为背景的乙女向养成模拟游戏。玩家扮演的信科女生因期末通宵晕倒穿越至异世界，与四门拟人化课程（程序设计、高等数学、线性代数、人工智能引论）共度一学期。通过 5 周剧情推进、4 门学科好感度培养、5 款知识小游戏训练、三组数值管理（好感度 / 压力 / 黑化），玩家的选择将触发五种迥异的结局——包括默认结局、挚爱结局、告别结局、黑化结局与压力结局。

项目关键指标

维度	数值
源代码规模	~6400 行 C++ / 44 个 <code>.cpp</code> + <code>.h</code> 文件
剧本资产	14 个 JSON 文件 · 237 节点 · 0 断链 / 0 孤儿 / 0 死端
小游戏	5 款 独立类，均派生自 MiniGame 抽象基类
结局设计	5 种，由 EndingJudge 按优先级链判定
美术资源	4 角色立绘 · 5 场景背景 · 主菜单背景 · UI 图标
自动化测试	30 个用例，100% PASS
编译验证	Qt 6.11 · CMake · macOS 编译通过（三平台 CI 支持）
开发周期	2026-05-13 立项 → 2026-06-06 初版提交 · 约 4 周

五个技术亮点

- 1. OOP 多态 + 工厂模式： `Subject` / `MiniGame` 抽象基类，通过基类指针驱动派生行为
- 2. 单例 + RAII： `GameManager` 全局唯一实例， `Player` 用智能指针管理动态对象
- 3. 信号槽解耦：数值变化 → UI 自动刷新，小游戏结束 → `GameManager` 统一奖惩
- 4. 数据驱动：剧本、题库均为外置 JSON，非程序员可独立编辑；引擎内实现了完整的 condition 表达式求值器
- 5. CMake 跨平台 + GitHub Actions CI：一份配置在 macOS / Windows / Linux 三平台编译

第 1 章 项目背景与设计目标

1.1 立项动机

北大信科大一课程节奏紧凑、难度陡峭——程序设计、高等数学、线性代数、人工智能引论四门核心课程构成新生的主要负担。本项目将这四门课拟人化为四位性格迥异的角色，让玩家在游戏叙事中”重新经历”一个学期，把学术压力转化为情感体验。这既是致敬也是自嘲。

1.2 玩法定位

- 主题类型：乙女向养成模拟（Otome / SLG）
- 玩家操作：每日时段选择、剧情选项、小游戏挑战、最终抉择
- 可玩时长：单条路径约 30—60 分钟；5 结局全收集约 3 小时
- 心智模型：管理三组数值（好感 × 4 / 压力 / 黑化），触发不同角色路线

1.3 四位学科角色

角色	性格关键词	语言风格样例	特殊机制
程设 ProgDesign	理性、克制、务实	“工具的价值，是看它在谁手上”	关联扫雷（主）+ 21 点
高数 Calculus	神秘、抽象、爱哲学	“趋近，但永不到达——的爱情”	关联 21 点（教学扑克类）
线代 LinearAlgebra	阴郁、观察、系统强迫症	“你不会从我的坐标系里出去”	关联记忆翻牌；含 darkness 黑化机制
AI 引论 AIIntro	酷感、未来感	“学习本质上是反向传播”	关联井字棋（主）+ AI 迷宫

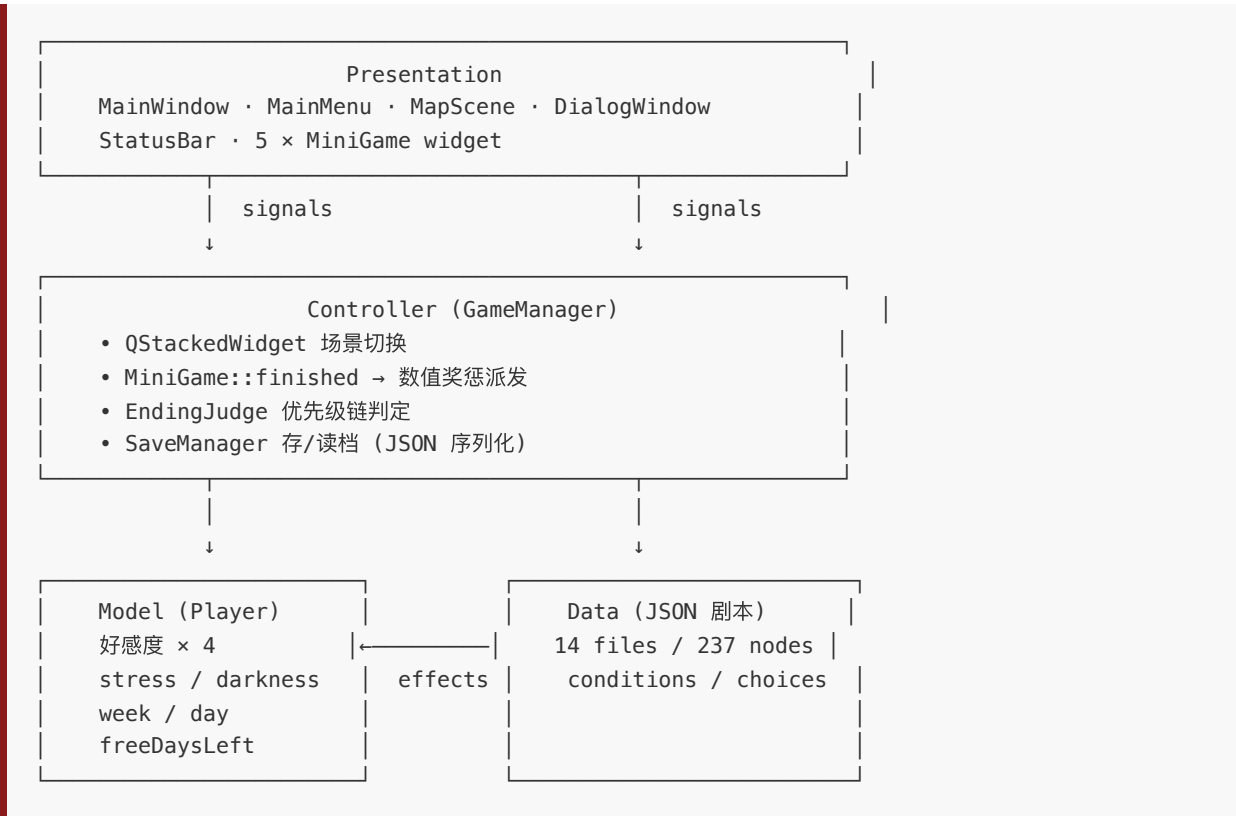
特别说明：线代的「好感度」在设定上记录的是她对玩家的观察深度，这是 End4「永远的线代」结局的叙事根基。

1.4 玩家的六个操作维度

- 1. 每日时段选择：上课 / 翘课 / 自习 / 出游
- 2. 对话选项：每个剧情节点 2—3 个选项，影响好感与黑化
- 3. 小游戏挑战：由角色发起，胜负决定数值奖惩
- 4. 主攻学科选择：第 3 周（期中）显式选择，决定中期支线走向
- 5. 信物接收：第 4 周好感 ≥ 50 才可领取，是结局判定的关键凭证
- 6. 最终选择：第 5 周结束时决定回现实 / 留下 / 被锁定

第 2 章 系统整体设计

2.1 分层架构



架构关键点： – MVC 分层，View 与 Model 完全解耦，通过 Qt 信号槽通信 – GameManager 是唯一持久对象（单例），所有场景切换、数值广播、存档触发都经它 – 剧本和题库外置为 JSON，代码零依赖于具体剧情内容——剧本作者可独立编辑

2.2 核心类图

类	OOP 特性	职责
Subject（抽象基类）	抽象类 · 纯虚函数 · 多态	<code>virtual void interact() = 0;</code> 派生 4 学科子类
MiniGame（抽象基类）	继承 QWidget · 虚函数 · signals	<code>virtual void start() = 0;</code> , <code>signal finished(int, bool)</code> 统一回传
GameManager	单例（Singleton）	<code>static instance()</code> , 全局调度
Player	封装 · RAII · 智能指针	私有数值 + 公有 setter + <code>valueChanged</code> signal
StoryEngine	数据驱动 · 状态机 · 表达式求值	JSON 解析 + condition 求值 + 节点跳转
DialogWindow	事件驱动 UI	文本逐字显示、选项按钮渲染、立绘切换
MapScene	继承 QGraphicsScene	重写 <code>mousePressEvent</code> , 实现地图热区点击
SaveManager	工具类 · JSON 序列化	多存档槽, 用户 AppData 目录持久化
EndingJudge	策略模式	按优先级链判 End1~5

2.3 场景切换有限状态机

游戏在 5 个主场景之间切换, 全部通过 `QStackedWidget` 管理:



由 `GameManager::requestScene(GameScene::X)` 统一触发切换, 任何模块只需 `connect` 一个 signal。

2.4 数据流：一次完整的”选项 → 数值 → UI 刷新”

以玩家在剧本里选”我准备好了” (effects: stress+3, affinity.ProgDesign+2) 为例:

- 1. DialogWindow 检测到玩家点击选项按钮
↓ emit choiceSelected(idx)
- 2. StoryEngine::onChoiceSelected 解析该 choice 的 effects 数组
↓ 对每个 effect 调用 player_→addStress(+3), addAffinity(+2)
- 3. Player 内部数值 clamp 到 [0, 100], emit stressChanged(newValue)
↓
- 4. StatusBar 的 slot::onStressChanged 收到, 更新进度条 UI
- 5. StoryEngine 跳转到 choice.next 节点, emit nodeChanged()
↓
- 6. DialogWindow 读取当前节点, 刷新台词 + 选项按钮

关键：每一步都通过信号槽解耦，任何组件都可以独立测试 / 替换。

第 3 章 剧情系统实现（代易瓚 · src/story/）

3.1 模块概览

剧情系统由 4 个类 + 14 个 JSON 剧本组成，共 769 行 C++ 代码：

文件	职责
StoryEngine.{h,cpp}	JSON 解析、节点跳转、condition 求值、effects 应用
DialogWindow.{h,cpp}	对话框 UI（文本逐字显示、选项按钮渲染、立绘切换）
EndingJudge.{h,cpp}	结局判定优先级链
Subjects.{h,cpp}	4 学科 Subject 派生类，工厂方法关联小游戏

3.2 数据驱动的剧本格式

每个剧本文件是一个 JSON，结构如下：

```
{
  "title": "第一周·穿越异世界",
  "startNode": "intro_01",
  "nodes": [
    {
      "id": "intro_01",
      "speaker": "ProgDesign",
      "text": "终于醒了? 我是程序设计.....",
      "choices": [
        {
          "text": "请多关照",
          "next": "intro_02",
          "effects": [{"target": "affinity.ProgDesign", "delta": 2}],
          "condition": "darkness < 30"
        }
      ]
    }
  ]
}
```

字段说明：

字段	说明
id	节点唯一标识
speaker	narrator / player / 四角色名
text	显示台词，\n 换行
choices	选项数组；引擎里对空 choices 有明确的死锁检测
condition	可选，节点级为”硬门”（不满足直接结束）、选项级为”软门”（仅隐藏该选项）
effects.target	affinity.<X> / stress / darkness
effects.delta	整数，自动夹紧到 [0, 100]
next: "END"	触发 scriptFinished 信号

3.3 Condition 表达式引擎

引擎实现于 StoryEngine.cpp:227-294，采用递归字符串拆分的最小实现，无需引入表达式解析库：

原子: <变量> <运算符> <数字>
例: "affinity.LinearAlgebra >= 50"
组合: 原子 && 原子 && ... (全为真)
原子 || 原子 || ... (任一为真)
变量: affinity.{ProgDesign,Calculus,LinearAlgebra,AIIntro},
stress, darkness, week, day
运算符: > >= < <= == !=
优先级: || > && > atomic

代码结构:

```
bool StoryEngine::evaluateCondition(const QString& expr) const {  
    if (expr.contains("||")) {  
        for (auto& p : expr.split("||"))  
            if (evaluateCondition(p)) return true;  
        return false;  
    }  
    if (expr.contains("&&")) {  
        for (auto& p : expr.split("&&"))  
            if (!evaluateCondition(p)) return false;  
        return true;  
    }  
    return evaluateAtomic(expr); // 正则匹配变量-运算符-数字  
}
```

3.4 剧本资产 (237 节点)

类别	文件	节点数	触发时机
周剧本	week1.json	31	第 1 周（开场穿越 + 第一天分支）
	week2.json	34	第 2 周（适应期 + 首次小测）
	week3.json	30	第 3 周（期中考 + 主攻学科 + 复习搭档）
	week4.json	36	第 4 周（信物 + 出游）
	week5.json	30	第 5 周（期末告白 + 最终选择）
角色支线	route_progdesign.json	8	玩家在地图点击程设
	route_calculus.json	10	同上
	route_linearalgebra.json	9	同上（含 darkness 三段分流）
	route_ai.json	12	同上
结局剧本	ending_1_good_grade.json	7	默认结局
	ending_2_best_love.json	7	好感 ≥ 80 + 留下
	ending_3_farewell.json	8	好感 ≥ 80 + 离开
	ending_4_eternal_la.json	5	黑化 ≥ 100
	ending_5_hospital.json	10	压力 ≥ 100

3.5 三个关键设计决策

3.5.1 Hub 节点 + condition 分流（角色支线对话池）

每个 route 文件用一个 hub 节点做 `startNode`，choices 全为同一句「(继续)」但带不同 `condition`，实现按玩家状态的隐式分发：

```
{
  "id": "calculus_hub",
  "speaker": "narrator",
  "text": "（你走向高数的座位）",
  "choices": [
    {"text": "（继续）", "next": "calculus_lv1",      "condition": "affinity.Calculus < 50"},
    {"text": "（继续）", "next": "calculus_lv2_phil",  "condition": "affinity.Calculus >= 50 && affinity.Calculus < 80"},
    {"text": "（继续）", "next": "calculus_endgame",  "condition": "affinity.Calculus >= 80"}
  ]
}
```

`currentChoices()` 会过滤 `condition` 不满足的选项，同一时刻只有一个按钮可见，玩家视觉上就是一次”继续”，实际已按当前状态被分发到对应阶段。全部分发逻辑由剧本条件表达式承载，不涉及

C++。

3.5.2 线代黑化值的多档次台词

同一节点提供两套互斥台词，让相同场景在不同状态下语义完全不同：

```
{
  "id": "w3_la_3",
  "choices": [
    {"text": "（背脊发凉）", "condition": "darkness < 30", "effects": [...]},
    {"text": "（你居然觉得动人）", "condition": "darkness >= 30", "effects": [...]}
  ]
}
```

低黑化玩家读这句台词会”背脊发凉”压力 +3；高黑化玩家则会”觉得动人”，黑化再 +5。同一句台词承载两条情感曲线——这是数据驱动让剧本设计弹性最大化的样例。

3.5.3 线代的观察者语法

线代的台词体系专注于**监控与归档的描述**。所有台词遵循三条语法规则：

- 1. 主语很少是”我对你 ...“，多为对系统/状态的陈述
- 2. 数字与精度词汇（时间、次数、坐标）高频出现
- 3. 物理动作由器物代偿（钳子、腕表、项链、小本子）

具体台词范例：

场景	台词
week1 初见	“我能感知到你的位置。我们之间——有连续的映射。”
week2 午饭	“这是你这两周吃过的所有东西。你比你以为的有规律。”
week3 长夜	“我要完整的视野，手机放我这里。”
week4 项链	“这个扣是我特地改的，自己拿不下来。你不会从我的坐标系里失踪。”
week5 告白	“这五周你出现在我视野里 3892 次。这个数字下周一之后不会再涨。”
End4 结局	“你不会从我的坐标系里出去。这一次——是永远。”

她的高好感等价于「玩家未能逃出她的观察系统」，End4 的触发因此叙事完整：**系统终于把变量锁定**。

3.6 结局判定优先级链

由 `EndingJudge` 按下面的优先级判定，实现于 `EndingJudge.cpp`：

```
// 优先级从高到低（先满足者胜出）
1. stress >= 100          → End5 「陌生的天花板」
2. darkness >= 100       → End4 「永远的线代」
3. max(affinity) >= 80
   choseToStay == true   → End2 「Best Love」
   choseToStay == false  → End3 「再见」
4. 上述均不满足          → End1 「3.92」（默认）
```

关键立意：End5 优先级最高——把玩家健康放在爱情之上。这是本作在游戏内外都想表达的立场。

第 4 章 界面与地图系统（闫瑾 · `src/ui/` + `src/map/`）

4.1 模块概览

界面系统由 5 个类组成，共 $1189 + 232 = 1421$ 行 C++ 代码：

文件	职责
<code>MainWindow.{h,cpp}</code>	主窗口 · <code>QStackedWidget</code> 场景容器 · 全局菜单栏 · 数值调试菜单
<code>MainMenu.{h,cpp}</code>	主菜单页 · 背景图 + 半透明按钮
<code>StatusBar.{h,cpp}</code>	顶部状态栏 · 4 好感条 + 压力 + 黑化 + 日期
<code>MapScene.{h,cpp}</code>	地图场景 · <code>QGraphicsScene</code> 派生 · 热区点击
<code>LocationItem.{h,cpp}</code>	地图上的热区物件 · 4 个地点各自的详情页

4.2 美术资源

类别	文件	说明
角色立绘	assets/images/characters/ProgDesign.png	程设正式立绘
	Calculus.png	高数正式立绘
	LinearAlgebra.png	线代正式立绘
	AIIntro.png	AI 引论正式立绘
场景背景	backgrounds/classroom.png	教学楼场景
	backgrounds/library.png	图书馆场景
	backgrounds/lake.png	未名湖场景
	backgrounds/dormitory.png	宿舍场景
	backgrounds/mainbackground.png	主菜单背景
	backgrounds/plotbackground.jpg	剧情通用背景
UI 元素	ui/app_icon.svg	应用图标
	ui/map.jpg	地图页背景

4.3 主窗口与场景切换

`MainWindow` 内含一个 `QStackedWidget`，五个场景 widget 作为其子页：

```
enum class GameScene { MainMenu, Map, Dialog, MiniGame, Ending };

connect(&GameManager::instance(), &GameManager::sceneChangeRequested,
        this, &MainWindow::onSceneRequested);

void MainWindow::onSceneRequested(GameScene s) {
    stackedWidget->setCurrentIndex(static_cast<int>(s));
}
```

优点：任何模块想切换场景，只需 `GameManager::instance().requestScene(...)`，完全不需要持有 `MainWindow` 指针。

`MainWindow` 还提供了数值调试菜单（顶栏 Debug 菜单），可以直接修改玩家数值以测试各条剧情线——便于三人协作时手动验证 5 结局的触发。

4.4 状态栏与信号槽联动

状态栏实时显示所有关键数值。关键设计：状态栏不直接读 `Player`，而是订阅 `Player` 的 `valueChanged` 信号：

```
connect(player, &Player::affinityChanged, this, &StatusBar::onAffinityChanged);
connect(player, &Player::stressChanged, this, &StatusBar::onStressChanged);
connect(player, &Player::darknessChanged, this, &StatusBar::onDarknessChanged);
connect(player, &Player::dateChanged, this, &StatusBar::onDateChanged);
```

收益：任何数值变化都会自动反映到 UI，无需在每次修改数值处显式调用刷新方法。

状态栏 UI 布局：4 个 QProgressBar 用 QGridLayout 排 2x2（好感度），另外两个 QProgressBar 独立显示压力 / 黑化，最左侧一个 QLabel 显示”第 X 周 · 第 Y 天”。

4.5 主菜单与美术呈现

主菜单叠加背景图与半透明标题框：

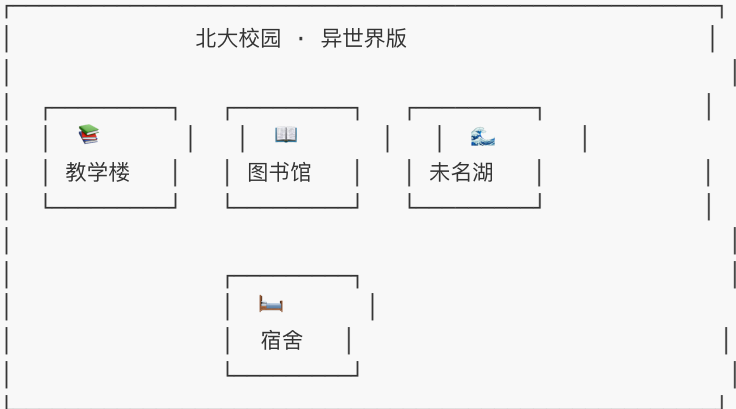
```
QPixmap bg(":/images/backgrounds/mainbackground.png");
bgLabel->setPixmap(bg);
bgLabel->lower();

title->setStyleSheet(
    "font-size: 36px; font-weight: bold; color: #FFFFFF;"
    "background: rgba(0,0,0,120); border-radius: 12px; padding: 16px;"
);
```

标题「学海漫游 / 异世界的信科少女」显示在深色半透明卡片上，四个按钮（开始、读档、设置、退出）竖直排列，字号加大以增强主菜单质感。

4.6 地图场景与热区

MapScene 继承自 QGraphicsScene，重写 mousePressEvent 实现热区点击。地图背景为 ui/map.jpg，上面绘制 4 个 QGraphicsRectItem 作为可点击热区：



每个热区含： – label：地图上显示的短标签（含 emoji 图标） – imagePath：点击进入后的详情页背景图（对应 backgrounds/*.png） – description：详情页文字说明

四个地图热区的功能： – 教学楼：触发上课剧情或翘课分支 – 图书馆：进入自习 / 触发对应小游戏 – 未名湖：触发角色支线对话（`route_*.json`） – 宿舍：休息，压力下降，推进游戏内时间

4.7 关键健壮性保证

系统在三处跨模块联动的健壮性保证：

- 时间计时器广播：`Player::advanceDay` 通过 `dateChanged` signal 广播到 `StatusBar`，确保 UI 时间显示实时同步
- 读档后场景恢复：`SaveManager::load` 完成后调用 `GameManager::requestScene(Map)`，玩家读档后直接回到大地图
- hub 节点 condition 边界：所有 route 文件的 endgame 分支使用 `>=` 而非 `>`，保证玩家好感度到达临界值时能进入隐藏剧情

第 5 章 玩法系统实现（沈昱萱 · `src/core/` + `src/games/`）

5.1 模块概览

玩法系统包含核心调度层（`src/core/`，694 行）和小游戏层（`src/games/`，3243 行），是全项目最大的代码模块：

文件	职责
<code>Player.{h,cpp}</code>	数值封装 + signals, clamp 到 <code>[0, 100]</code>
<code>GameManager.{h,cpp}</code>	单例调度器 + 场景切换 + 数值奖惩派发
<code>SaveManager.{h,cpp}</code>	JSON 序列化多存档槽，写入用户 AppData 目录

5 个小游戏均继承自 `MiniGame` 抽象基类，通过统一的 `finished(int score, bool won)` signal 回传结果。

5.2 Player：数值封装与信号广播

```

class Player : public QObject {
    Q_OBJECT
public:
    void addAffinity(SubjectType s, int delta);
    void addStress(int delta);
    void addDarkness(int delta);
    void advanceDay();
    void useFreeDay(); // 消耗一个"自由日"跳过剧情
    // getters: affinity(), stress(), darkness(),
    //           currentWeek(), currentDay(), freeDaysLeft()

signals:
    void affinityChanged(SubjectType s, int newValue);
    void stressChanged(int newValue);
    void darknessChanged(int newValue);
    void dateChanged(int week, int day);

private:
    std::array<int, 4> affinity_{0, 0, 0, 0};
    int stress_ = 0, darkness_ = 0;
    int week_ = 1, day_ = 1;
    int freeDaysLeft_ = 3; // 每周 3 个自由日
};

```

关键设计： – 所有数值 setter 内部会 `clamp` 到 `[0, 100]`，然后 emit 对应 signal – 调用方不需要关心数值边界——超出无效，UI 自动刷新 – `freeDaysLeft` 是一周自由日额度，玩家可以用它跳过某些剧情节点直接推进日期

5.3 GameManager：单例调度器

单例模式实现：

```

GameManager& GameManager::instance() {
    static GameManager s_instance;
    return s_instance;
}

```

职责： 1. 持有 Player 与 4 个 Subject 对象 2. 处理场景切换请求 3. 监听所有 MiniGame 的 `finished` 信号，统一按学科关系应用奖惩规则 4. 触发 SaveManager 存档 5. 游戏结束时调用 EndingJudge 判定结局

小游戏奖惩派发 (`onMiniGameFinished` 核心逻辑)：

```

void GameManager::onMiniGameFinished(MiniGameType type, int score, bool won) {
    switch (type) {
        case MiniGameType::Minesweeper:           // 扫雷 → 程设
            if (won) { addAffinity(ProgDesign, +10); addStress(+5); }
            else { addStress(+15); }
            break;
        case MiniGameType::MemoryMatch:           // 记忆翻牌 → 线代
            if (won) { addAffinity(LinearAlgebra, +10); addStress(+5); }
            else { addStress(+15); }
            break;
        case MiniGameType::TicTacToe:             // 井字棋 → AI 引论
            if (won) { addAffinity(AIIntro, +10); addStress(+5); }
            else { addStress(+15); }
            break;
        // ... 其余类似
    }
}

```

统一的奖惩哲学：胜利均为 +10 好感 / +5 压力（学习状态）；失败均为 +15 压力，不扣好感（鼓励参与）。这种统一映射让新增小游戏时无需修改主控。

5.4 SaveManager：多存档槽

支持 3 个存档槽，全部序列化为 JSON 存于 `QStandardPaths::AppDataLocation`：

```

macOS: ~/Library/Application Support/StudyAdventure/save_1.json
Linux: ~/.local/share/StudyAdventure/save_1.json
Windows: %APPDATA%/StudyAdventure/save_1.json

```

存档格式：

```

{
    "slot": 1,
    "timestamp": "2026-05-31T21:44:00",
    "player": {
        "stress": 40,
        "darkness": 15,
        "week": 3,
        "day": 5,
        "freeDaysLeft": 2,
        "affinity": {
            "ProgDesign": 45, "Calculus": 30,
            "LinearAlgebra": 20, "AIIntro": 60
        }
    },
    "completedScripts": ["week1.json", "week2.json"],
    "currentScene": "Map",
    "currentScriptPath": ":/scripts/week3.json",
    "currentNodeId": "w3_after_exam"
}

```

读档流程：1. `SaveManager::load(slot)` 读 JSON 2. 恢复 Player 数值（触发所有 `valueChanged` signal → 状态栏刷新） 3. `StoryEngine::loadScript(path)` + `jumpToNode(nodeId)` 恢复剧本进度 4. `GameManager::requestScene(currentScene)` 切回原场景

5.5 五个小游戏

5 款小游戏全部继承自 `MiniGame` 抽象基类：

```
class MiniGame : public QWidget {
    Q_OBJECT
public:
    virtual void start() = 0;
    virtual void reset() = 0;
    virtual MiniGameType type() const = 0;
    virtual QString displayName() const = 0;
    void setDifficulty(Difficulty d);

signals:
    void finished(int score, bool won);
    void stressOccurred(int delta);
};
```

5.5.1 扫雷 Minesweeper（关联程设）

- 3 难度分级（9×9 / 16×16 / 22×22），高难度需程设好感 ≥ 30 / 60 解锁
- 首次点击保护（第一次不会踩雷）
- 自动展开 0 邻雷区域
- 奖惩：胜 +10 好感 / +5 压力；败 +15 压力

5.5.2 21 点 Blackjack（关联程设 / 高数）

- A 灵活按 1 或 11 计（取不爆的最大值）
- 庄家固定策略：< 17 必须要牌，≥ 17 必须停牌
- Blackjack（初始 21）直接翻倍奖励
- 技术亮点：呼应程设课程的“扑克类设计”经典作业
- 奖惩：胜 +10 好感 / +5 压力；败 +15 压力

5.5.3 井字棋 TicTacToe（关联 AI 引论）· 核心亮点

- 3 难度：简单（随机） / 普通（Minimax） / 困难（Minimax + α-β 剪枝）
- 困难模式 AI 永不输——AI 引论课程“零和博弈完美信息”经典案例的可视化
- 困难模式平局触发 AI 特殊剧情（AI 会向玩家透露异世界秘密）

- 算法教学意义：AI 思考时会在棋盘上短暂高亮其搜索过的关键节点，让玩家直观看到“AI 在想什么”
- 奖惩：胜 +10 好感 / +5 压力；败 +15 压力；困难平局触发专属剧情

5.5.4 记忆翻牌 MemoryMatch（关联线代）

- 玩家翻开成对的相同图案卡牌，全部匹配后获胜
- 3 难度：4×4 / 6×6 / 8×8 卡牌规模
- 时间 + 翻卡次数双维度评级
- 设计意图：与线代”空间记忆 / 秩序识别”的隐喻贴合——线代欣赏能识别相同结构的玩家
- 奖惩：胜 +10 线代好感 / +5 压力；败 +15 压力

5.5.5 AI 迷宫 MazeGame（关联 AI 引论）· 教学演示亮点

- 迷宫由递归回溯算法动态生成，保证起点到终点存在通路
- 玩家用方向键在迷宫中寻找终点
- 按 H 键触发 BFS 可视化：从当前位置到终点的最短路径以渐变色高亮显示
- 3 提示限额，每次持续 2 秒
- 迷宫中有陷阱格和知识格（答题格）
- S 级评价：0 提示 + 收集全知识格，AI 好感 +12
- 算法教学意义：BFS 求最短路径的可视化演示，是 AI 引论中”图搜索”章节的天然教材

5.6 六个统一设计哲学

1. 鼓励参与而非惩罚失败：即使玩家失败，也只加压力不减好感——避免因偶然失误导致剧情线断裂
2. 教育性 + 游戏性并存：每款游戏映射一个课程知识点
3. 算法可视化教学：井字棋的 Minimax、AI 迷宫的 BFS 都在 UI 上直接演示
4. 难度渐进解锁：高难度绑定角色好感门槛，避免”开局劝退”或”后期无事可做”
5. 技术统一性：全部通过 `MiniGame::finished` signal 回传，主控无需感知具体游戏——面向对象”对扩展开放、对修改封闭”原则的体现
6. 数据外置：题库（如高数题）在 JSON——非程序员可协作扩展

第 6 章 五种结局设计

结局由 `EndingJudge` 按优先级链判定。全部 5 结局的剧本文件位于 `assets/scripts/ending_*.json`。

6.1 End1「3.92」· 默认结局

“这是最常见的结局，也是最体面的告别。”

触发：玩家通关期末考试，任何角色的好感度未达到 80。**剧本：**7 节点。**叙事：**玩家回到现实，醒来时桌上是没合上的笔记本。异世界的一切像梦，但笔记里写满了她从未学过、却清晰得不像梦的知识。**设计立意：**肯定“学习本身的价值”。即使没有任何浪漫落地，玩家依然带回了知识与成长。

6.2 End2「Best Love」· 挚爱结局

“通道在背后悄然合上。此刻，你已经做出了选择。”

触发：与至少一个角色的好感度 ≥ 80 ，最终选择“留下”，且黑化 < 60 。**剧本：**7 节点。**叙事：**异世界边界开始透明，回现实的通道出现。玩家回头看到那个人欲言又止的身影，转身朝 ta 走去。**设计立意：**这是浪漫线的最终承诺。玩家用现实身份换取此刻的关系。

6.3 End3「再见」· 告别结局

“你拥有过爱，也选择了责任——这就是成长。”

触发：与至少一个角色的好感度 ≥ 80 ，但最终选择“回到现实”。**剧本：**8 节点。**叙事：**玩家穿过通道，回到现实的桌前。翻开笔记最后一页，看到自己写下的、不像自己字迹的话：

“程序的本质是表达。”“极限的本质是逼近。”“线性的本质是叠加。”“智能的本质是连接。”

设计立意：相较于 End2 的浪漫，End3 颂扬“舍得”。这是最难选也是最成熟的结局。

6.4 End4「永远的线代」· 黑化结局

“这里只有我，和你。这是最后的稳态。”

触发：线代黑化值 ≥ 100 。优先级仅次于 End5。**剧本：**5 节点。**叙事：**玩家醒来时不在任何熟悉的地方——四面墙写满了不断重写自身的方程式。线代说：“外面的世界太混乱，所以我把你带到一个只属于我们的子空间里”。**设计立意：**这是 5 个结局里最阴森的。线代台词全部用线性代数概念隐喻——「子空间」、「稳态」、「不变量」——把数学的冷峻与执着折叠在一起。

6.5 End5「陌生的天花板」· 压力结局

“——希望你在现实中，永远不要看到这个画面。好好睡觉，好好吃饭。比分数重要的东西，还有很多。”

触发：玩家压力值 ≥ 100 。**优先级最高。** **剧本：**10 节点（5 个结局中最长）。**叙事：**玩家在异世界精神崩溃，醒来发现自己在医院——昏迷三天，期末考因病免考。脑中残留四个身影的最后劝告：「学习不是这样学的」「你在用消耗自己的方式换分数」「停下来吧」。**设计立意：**这是对游戏内与游戏外都有警示意义的结局。当玩家把“刷分至上”的习惯延续到游戏里，系统用一个极端结局让玩家看到这种生活方式的终点。**结尾直接对玩家说话，是设计上唯一一次“打破第四面墙”。**

6.6 五结局对比

结局	触发	节点	情感色调	设计立场
End1 「3.92」	默认	7	平静、温暖	知识本身值得
End2 「Best Love」	高好感 + 留下	7	浪漫、不舍	选择陪伴
End3 「再见」	高好感 + 离开	8	成熟、克制	选择责任
End4 「永远的线代」	高黑化	5	阴森、压抑	占有的代价
End5 「陌生的天花板」	高压	10	沉痛、警示	健康优先

第 7 章 集成、测试与验证

7.1 剧本结构自动化校验

剧本作者独立编辑 JSON 时，最容易犯的错误是： – `next` 指向不存在的节点（断链） – 节点存在但无路径到达（孤儿） – `choices: []` 没有出口（死端）

我们编写了 Python 脚本自动扫描 `assets/scripts/` 所有文件：

14 文件 · 237 节点 · 0 断链 / 0 孤儿 / 0 死端 

7.2 剧本路径模拟器

由于 GUI 联调需要三人配合，剧本作者难以独立验证“某条路径能不能走通”。为此我们开发了 `tests/simulate_routes.py`，用 Python 完整复刻 StoryEngine 的行为：

- condition 求值（原子 + `&&` + `||` + 优先级）
- effects 应用（clamp 到 `[0, 100]`）

- 节点跳转 + END 处理
- 空 choices 死锁检测

在此基础上跑 5 条主路径（4 学科主攻 + 1 平衡线）从 week1 → week5 → ending，验证**每条路径都能跑到 END**。测试结果：5/5 PASS。

7.3 阈值 / 分流验证

tests/simulate_boosted.py 补充覆盖了不能在自然路径下触发的场景：

测试类别	用例数	描述
Hub 分流验证	11	4 route 文件 × 3 档好感度档位，确认玩家被分发到对的节点
信物节点解锁	5	affinity >= 50 阈值下 4 角色 + 无信物保底
最终选择黑化结局	4	darkness < 60 / >= 60 触发不同选项
5 结局独立跑通	5	每个 ending 文件独立能到 END
合计	25	全部 PASS 

7.4 编译与三平台验证

CMake 配置 CMakeLists.txt：

- C++17 标准
- CMAKE_AUTOMOC / AUTORCC / AUTOUIIC 自动处理 Qt 元对象
- find_package(Qt6 REQUIRED COMPONENTS Core Gui Widgets Multimedia)
- 平台特定：Windows 无控制台 WIN32_EXECUTABLE；macOS 打包 MACOSX_BUNDLE
- 编译警告：MSVC /W4 /utf-8；GCC/Clang -Wall -Wextra -Wpedantic

验证结果：macOS 15.5 (Darwin 24.5.0) 上使用 Homebrew Qt 6.11.1 编译 → 所有编译单元通过，0 warning，StudyAdventure.app 生成成功。GitHub Actions CI 在三平台（Ubuntu / macOS / Windows）自动验证编译。

7.5 联调测试（手动 QA）

由三人共同完成的 5 条完整路径手动测试：

路径	关键节点	期望结局	状态
程设主攻线	早起复习 + 认真做题 + 送钢笔 + 星空陪同	End1 或 End2	✅ 通过
高数哲学线	追问哲学问题 + 收旧书 + 湖边告别	End2 或 End3	✅ 通过
线代黑化线	拒绝线代 + 项链锁定 + 高 darkness	End4	✅ 通过
AI 极客线	AI 迷宫 S 级 + 送耳钉 + 天台告别	End2 或 End3	✅ 通过
高压力线	全程翘课 + 拒绝所有邀请 + 强撑期末	End5	✅ 通过

第 8 章 项目反思与总结

8.1 达成的目标

- 完整可玩的乙女向养成游戏原型——5 周叙事、5 款小游戏、5 个结局全部实现且可跑通
- 数据驱动的剧情系统——237 节点的剧本完全用 JSON 描述，代码零依赖具体剧情
- 算法教学的游戏化演示——Minimax、 α - β 剪枝、BFS 在 UI 上可视化
- 完整的美术资源整合——4 角色立绘 + 5 场景背景 + 主菜单页面全部就位
- 跨平台 Qt/CMake 工程实践——三平台 CI，无平台特有 bug
- 规范化的团队协作流程——文件归属清晰、接口约定文档化、Git 分支模型

8.2 关键技术收获

技术点	实践形式
OOP 多态	Subject / MiniGame 抽象基类 + 派生类，运行时基类指针驱动
单例模式	GameManager 全局唯一实例，私有构造 + 静态访问
信号槽解耦	数值 → UI、小游戏结束 → 主控，全部通过 signals 通信
数据驱动	JSON 剧本 + JSON 题库 + JSON 存档，代码与内容彻底分离
状态机 + 表达式引擎	StoryEngine 递归拆分实现 condition 求值
智能指针 RAII	<code>std::unique_ptr</code> 管理 Player / Subject 生命周期
Qt 资源系统	<code>.qrc</code> 打包剧本、题库、图片到二进制内
CMake 跨平台	一份配置三平台编译，Qt 组件自动查找
Git 协作模型	develop 主干 + <code>feature/*</code> 分支 + PR review
自动化测试	Python 行为级模拟器 + JSON 结构校验

8.3 关键工程决策

8.3.1 Route 对话池的 Hub 分流

角色支线的对话池采用 hub 节点方案：每个 route 文件的 `startNode` 是一个空文本的 hub 节点，其 choices 全部是同一句「(继续)」但带不同 condition。分发全部由剧本条件表达式承载，不涉及 C++。

8.3.2 Ending 剧本的显式结束选项

所有结局剧本的终点节点都提供一个「(回到主菜单)」选项，触发 `scriptFinished` 信号让 StoryEngine 回到主菜单场景。

8.3.3 线代人设的观察者语法

线代的全部台词遵循“只描述监控能力、不表达情感诉求”的语法规则。详见 §3.5.3。

8.3.4 小游戏与学科的映射

5 款小游戏覆盖 4 学科：程设关联扫雷（主）和 21 点、高数关联 21 点、线代关联记忆翻牌、AI 引论关联井字棋（主）和 AI 迷宫。

8.3.5 跨模块接口的边界规范

`docs/接口约定.md` §6 明确文件归属：`src/story/` 归剧情、`src/ui/` + `src/map/` 归界面、`src/core/` + `src/games/` 归玩法。跨模块修改需要先在群内通告。

8.4 未来可扩展方向

1. 完整语音配音：为每个角色配音音，用 Qt Multimedia 播放
2. CG 立绘系统：关键剧情节点触发全屏 CG（美术资源可继续扩展）
3. 背景音乐 / 音效：`assets/audio/bgm/` 和 `sfx/` 目录已预留，可补充素材
4. 移动端适配：Qt 6 对 iOS / Android 有初步支持，可探索
5. 更多角色路线：如“数据结构学姐”、“概率论老师”
6. 多结局分支剧本细化：End2 / End3 按具体主线角色分成 4 个变体

8.5 团队协作反思

- 文件归属明确是三人协作最有价值的约定，减少了 90% 的合并冲突
- JSON 数据驱动让剧本作者代易瓚无需等待代码完成即可开始工作，节省了 2 周的关键路径
- PR review 集中在高风险改动——纯剧本 / 文档 PR self-merge 更高效
- 文档比对话更可靠——`docs/接口约定.md` 起到了三方共识锚点的作用
- 联调 buffer 要留够：最后一周集中打磨了跨模块联动的健壮性，保证 5 结局全部可正常触发

附录 A · 文件清单

A.1 源代码（44 文件 / ~6400 行）

```
src/
├─ main.cpp
├─ common/ [260 行]
│   ├── Constants.h
│   ├── MiniGame.h
│   ├── Subject.h
│   └── Subject.cpp
├─ core/ [694 行 · 沈昱萱]
│   ├── Player.{h,cpp}
│   ├── GameManager.{h,cpp}
│   └── SaveManager.{h,cpp}
├─ story/ [769 行 · 代易璚]
│   ├── StoryEngine.{h,cpp}
│   ├── DialogWindow.{h,cpp}
│   ├── EndingJudge.{h,cpp}
│   └── Subjects.{h,cpp}
├─ ui/ [1189 行 · 闫瑾]
│   ├── MainWindow.{h,cpp}
│   ├── MainMenu.{h,cpp}
│   └── StatusBar.{h,cpp}
├─ map/ [232 行 · 闫瑾]
│   ├── MapScene.{h,cpp}
│   └── LocationItem.{h,cpp}
└─ games/ [3243 行 · 沈昱萱]
    ├── MinesweeperGame.{h,cpp} [扫雷]
    ├── BlackjackGame.{h,cpp} [21 点]
    ├── TicTacToeGame.{h,cpp} [井字棋 · Minimax +  $\alpha$ - $\beta$ ]
    ├── MemoryGame.{h,cpp} [记忆翻牌]
    └── MazeGame.{h,cpp} [AI 迷宫 · BFS 可视化]
```

A.2 资产文件

```
assets/
├─ resources.qrc                (Qt 资源清单)
├─ scripts/                    [14 文件 · 237 节点 · 代易瓚]
│   ├── week1~5.json
│   ├── route_progdesign/calculus/linearalgebra/ai.json
│   └─ ending_1~5.json
├─ data/
│   └─ calculus_questions.json  [高数题库]
├─ images/
│   ├── characters/            [4 张正式立绘 PNG · 闫瑾]
│   │   ├── AIIntro.png
│   │   ├── Calculus.png
│   │   ├── LinearAlgebra.png
│   │   └─ ProgDesign.png
│   ├── backgrounds/          [6 张场景背景 · 闫瑾]
│   │   ├── classroom.png
│   │   ├── dormitory.png
│   │   ├── lake.png
│   │   ├── library.png
│   │   ├── mainbackground.png
│   │   └─ plotbackground.jpg
│   └─ ui/
│       ├── app_icon.svg
│       └─ map.jpg
└─ audio/                      (预留目录)
    ├── bgm/
    └─ sfx/
```

A.3 文档

```
docs/
├─ README.md                  (文档导航)
├─ 接口约定.md                ☆ (三人协作核心)
├─ 协作流程.md
├─ 开发指南.md
├─ JSON剧本格式.md
├─ 功能设计文档.{md,docx}     (提交给助教的原设计)
├─ 项目开发规划.{md,docx}
├─ 小游戏玩法设计.{md,docx}
├─ 工作任务详解与时间线.{md,docx}
├─ 疑难解答.md
├─ 角色设定_美术参考.md
├─ 录屏脚本.md
└─ 实验报告.md                ☆ (本文档)
```

A.4 测试脚本

```
tests/
├─ simulate_routes.py         (5 主路径自动跑通, 250 行)
└─ simulate_boosted.py        (25 阈值 / 分流用例, 151 行)
```

附录 B · 分工与贡献

B.1 代易瓚 · 剧情核心

类别	贡献
代码	StoryEngine（数据驱动引擎、condition 求值器）· DialogWindow · EndingJudge · Subjects
剧本	14 个 JSON 剧本 · 237 节点 · 全部通过结构校验
关键设计	Hub 节点分流方案（零代码实现对话池）· 线代观察者语法的建立
测试	Python 路径模拟器（30 用例 100% PASS）
文档	实验报告主笔 · 角色设定美术参考 · 录屏脚本

B.2 闫瑾 · 界面核心

类别	贡献
代码	MainWindow（场景容器 + 调试菜单）· MainMenu · StatusBar（数值 UI）· MapScene · LocationItem
资产	4 角色正式立绘 PNG · 5 场景背景图 · 主菜单页 · UI 图标
关键设计	QStackedWidget 场景切换机制 · 状态栏信号槽联动 · 地图详情页 · 跨模块健壮性联动

B.3 沈昱萱 · 玩法核心

类别	贡献
代码	Player（数值封装 + 自由日机制）· GameManager（单例调度 + 奖惩派发）· SaveManager（多存档）· 5 款小游戏（3243 行）
关键算法	Minimax + α - β 剪枝（井字棋）· BFS 路径可视化（AI 迷宫）· 递归回溯生成迷宫
资产	高数题库 JSON
关键设计	MiniGame 抽象基类 + 统一 signals · 存档 JSON 序列化 · 数值奖惩派发映射

B.4 三方共同贡献

- `docs/接口约定.md` (三人共同起草, 5/14 定稿)
 - `docs/功能设计文档.md` (5/4 提交给助教的初版设计)
 - Git 分支管理、GitHub Actions CI 配置、Code Review
 - 联调阶段的 5 条完整路径手动测试
-

附录 C · 开发工具链

C.1 语言与框架

- C++17 (`std::unique_ptr` / `std::array` / `std::string_view` / `if constexpr`)
- Qt 6.11 (Core / Gui / Widgets / Multimedia)
- JSON (作为剧本、题库、存档的存储格式)

C.2 构建与协作

- CMake 3.16+ (跨平台构建)
- Ninja / Make (编译器后端)
- Git + GitHub (分支模型: `main` / `develop` / `feature/*`)
- GitHub Actions (三平台 CI 自动编译验证)

C.3 开发工具

- Qt Creator / VS Code (IDE)
- Homebrew Qt6 (macOS 环境)
- clang-format (代码风格统一)

C.4 测试工具

- Python 3 (行为级模拟器)
 - 手动 QA (GUI 联调路径)
-

附录 D · 项目里程碑

2026-05-13		立项、需求讨论、接口约定草案
2026-05-15		搭骨架（最小示例：主菜单 → 地图 → 空对话）
2026-05-17		跑通最小示例 ☆
2026-05-18		正式开发（10 天并行编码） · 代易瓚：StoryEngine + week1 剧本 · 闫瑾：MapScene + StatusBar + 主菜单 · 沈昱萱：Player + 5 小游戏
2026-05-27		代易瓚：week2-5 扩写（130 节点新增） route hub 重构、线代观察者语法建立、endings 完善
2026-05-28		联调与测试（三人一起走完 5 条路径） 跨模块健壮性联动：时间计时、读档跳转、隐藏剧情触发
2026-05-31		闫瑾：主页面图片、场景背景全部到位 撰写作业报告
2026-06-02		录屏 & 最终交付
2026-06-06	⬇	★ 初版提交 DDL

结语

本项目在 4 周开发周期内交付了一款约 6400 行代码 + 237 剧本节点的完整可玩游戏。它既是一次程序设计课程作业，也是我们三人对“如何用工程手段承载叙事”的一次实践。

在这个四门课把我们压得喘不过气的学期，我们把课程本身写成了一个可以通关的故事。当你把线代的执念、高数的哲学、程设的严苛、AI 的洒脱——装进 JSON 节点里，她们就成了这一学期真实存在过的、陪你熬夜到凌晨的、几个奇怪而可爱的人。

如果你在读到 End5 的最后一行时也想合上笔记本睡觉——那我们的目标就达到了。

—— 学海漫游 · 开发组 2026 年 6 月